

แผ่นสะท้อนแบบ Full Array

1. วัตถุประสงค์

1. เพื่อศึกษาการจำลองที่เหมือนแผ่นสะท้อนจริงที่ใช้ในการวิจัย โดยครอบคลุมโครงสร้างที่ประกอบด้วย Unit Cell หลายชุดจัดเรียงเป็น Array ที่ใช้ Frequency Domain Solver และ Time Domain Solver ในการจำลอง
2. เพื่อหาวิธีการจำลองในโปรแกรม CST ที่เหมาะสมกับคุณสมบัติของเครื่องมือและใช้ทรัพยากรคอมพิวเตอร์อย่างเหมาะสม
3. เพื่อสร้างสมการทำนายเวลาและทรัพยากรที่ใช้ในการรัน Simulation สำหรับ Full Array ขนาดต่างๆ ล่วงหน้า

2. อุปกรณ์ที่ใช้ในการทดสอบ

การประมวลผลแบบจำลองทั้งหมด ดำเนินการบนเครื่องคอมพิวเตอร์แล็ปท็อป Acer Aspire Go 15 (รุ่น AG15-71P-56AJ) โดยมีรายละเอียดคุณสมบัติทางฮาร์ดแวร์และซอฟต์แวร์ที่ใช้เป็นมาตรฐานในการทดสอบ ดังนี้:

รายการ	รายละเอียด
เครื่อง	Acer Aspire Go 15 (AG15-71P-56AJ)
ระบบปฏิบัติการ	Windows 11 Home Single Language
CPU	Intel Core i5-13420H (13th Gen), 8 Core / 12 Thread, 2.10 GHz
RAM	16 GB DDR5 (รองรับสูงสุด 32 GB)
Storage	512 GB SSD M.2 PCIe NVMe Gen 3.0



ที่มา: <https://www.acer.com/th-th/laptops/aspire/aspire-go-intel/pdp/NX.J4GST.001#pdpOverview>

3. Frequency Domain Solve

ในการใช้ Frequency domain solve เป็นตัวเลือกหนึ่งที่สามารถจำลองแผ่น Full Array และได้ผลลัพธ์ที่ต้องการ ได้แก่ S-Parameter (S_{11} , S_{21}) ของ Full Array ในย่านความถี่ 15–45 GHz รวมถึงสมการที่สามารถทำนายเวลาและหน่วยความจำที่ต้องใช้ได้ล่วงหน้า ซึ่งมีแผนการดำเนินงานดังนี้

1. ศึกษาคุณสมบัติของ Frequency Domain Solver
2. กำหนดค่า Setting ของ Frequency Domain Solver และรัน Model ตั้งแต่ Unit Cell จนถึง 5×5
3. บันทึก Tetrahedrons, RAM, เวลา ทั้งแบบมีและไม่มี Adaptive Mesh Refinement
4. วิเคราะห์ความสัมพันธ์และสร้างสมการทำนาย Tetrahedrons, เวลา, และ RAM (Dynamic Calibration)
5. Verification: เปรียบเทียบค่าที่ทำนายกับค่าจริงจากการรัน Simulation

3.1 หน้าทีและโครงข่ายที่ใช้(Mesh Type)

หน้าที่หลักของตัวแก้สมการนี้คือการคำนวณหาค่าพารามิเตอร์ S (S-parameters) [4] โดยโครงข่ายหลักที่ใช้ในการวิเคราะห์จะเป็นโครงข่ายเชิงปริมาตรแบบทรงสี่หน้าคือ Tetrahedral mesh [2]

3.2 ความสัมพันธ์ของเวลาและทรัพยากร

1. จำนวนความถี่ การคำนวณ 1 จุดความถี่ จะต้องใช้การรันsimulationใหม่ 1 รอบเสมอ ทำให้เวลาที่ใช้แปรผันตรงกับจำนวนจุดความถี่ วิธีนี้จึงทำงานได้รวดเร็วที่สุดเมื่อต้องการหาคำตอบเพียงจุดเดียวหรือจำนวนน้อยๆ [4]
2. ขนาดของโมเดล ทรัพยากรหน่วยความจำและเวลาที่ใช้ประมวลผลจะเพิ่มขึ้นแบบทวีคูณ(scale exponentially) ตามขนาดของโมเดล[3]

3.3 ความเหมาะสมสำหรับการนำไปใช้งาน

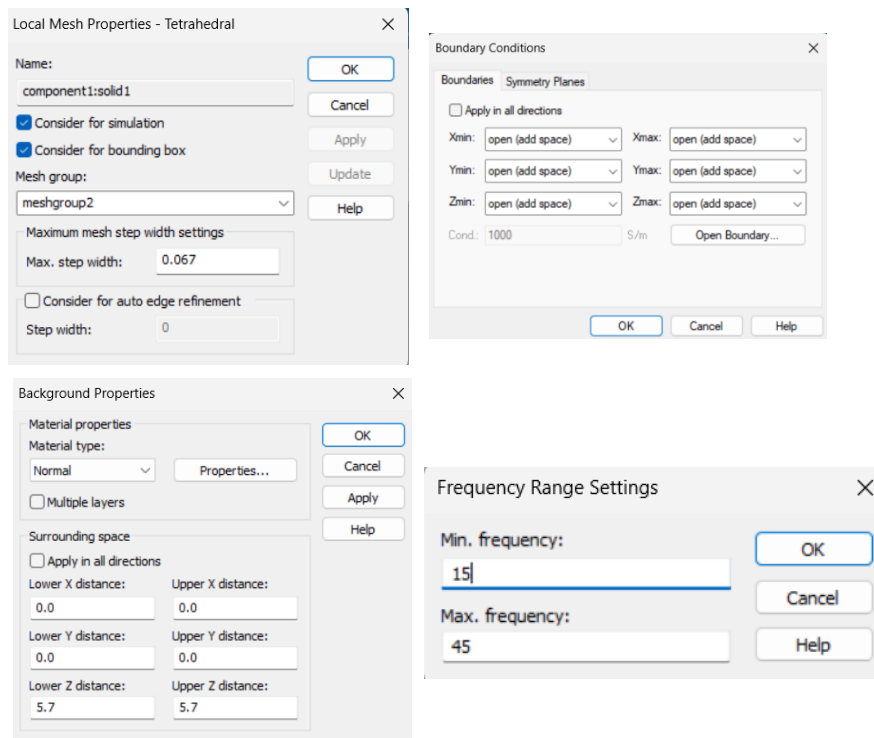
- เหมาะสำหรับ โครงสร้างที่มีขนาดเล็กทางไฟฟ้า(เช่น การออกแบบระดับ 1 unit cell) หรือชิ้นส่วนย่อยที่ต้องการความแม่นยำสูงๆ [2]
- ไม่เหมาะสำหรับ โครงสร้างขนาดใหญ่ที่มีความซับซ้อนสูง เช่น สายอากาศแบบ Full Array เพราะเกิดการใช้ทรัพยากรแบบทวีคูณจะทำให้คอมพิวเตอร์ทำงานไม่ไหว [3]

3.4 เทคนิคที่ช่วยเพิ่มประสิทธิภาพที่ใช้

- Adaptive Mesh Refinement มีระบบหาความคลาดเคลื่อน (Error) และทำการเพิ่มความหนาแน่นของตารางโครงข่ายเฉพาะจุดที่มีความชันของสนามแม่เหล็กไฟฟ้าสูงๆ ให้โดยอัตโนมัติ เพื่อให้ได้ผลลัพธ์ที่ลู่เข้าสู่ความแม่นยำสูงสุด [2]

3.5 ค่า Setting ในโปรแกรม CST

- Frequency Range: 15-45 GHz
- Boundaries: X,Y,Z(min/max) open(add space)
- Fraction of wavelength = 4 (default)
- Cell per wavelength (model/background) = 4/4 (default)
- Cell per max model box (model/background) = 10/1 (default)
- local mesh refinement ของตัวนำ (copper)= 0.067
- Z distance (lower/upper) = 5.7

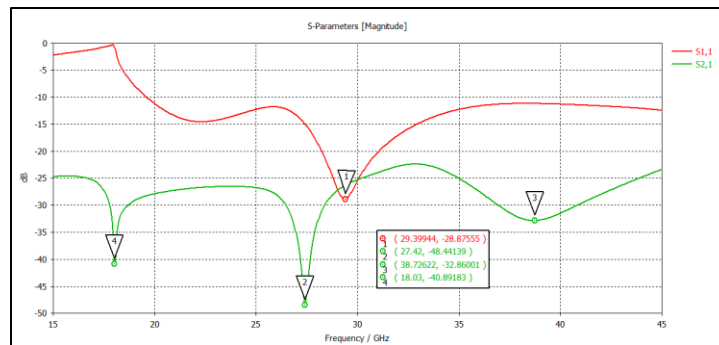


รูปที่ 4.1 ค่า Setting

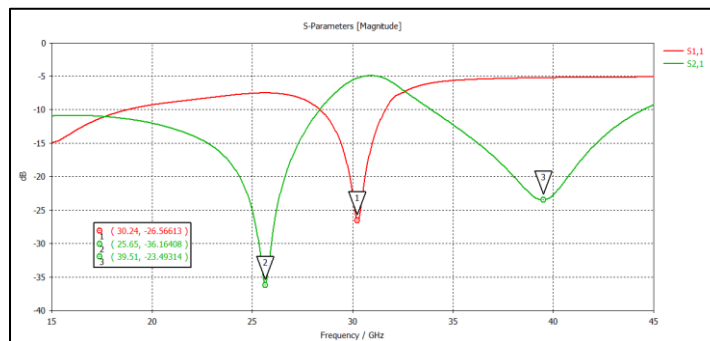
3.6 ผลการจำลองที่ได้ แบบที่ไม่ใช้ Adaptive Mesh Refinement

ในการคำนวณเวลาที่ใช้ในการรันsimulation และหน่วยความจำที่ใช้(RAM) เป็นค่าสูงสุดเมื่อทำการจำลอง ณ ขณะนั้น

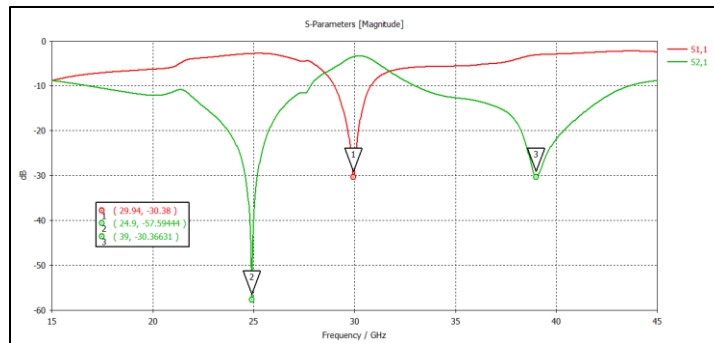
Model	Tetrahedrons	RAM	Time
Unit Cell	33,098	51%	38 วินาที
2x2	113,946	64%	3 นาที
3x3	244,285	68%	12 นาที
4x4	443,309	89%	26 นาที
5x5	686,401	94%	1 ชั่วโมง 13 นาที



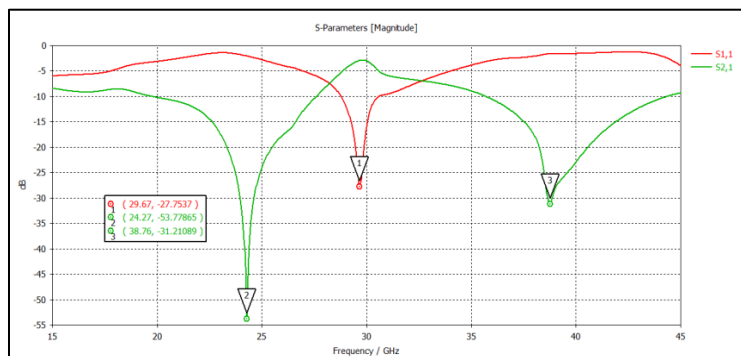
รูปที่ 4.2 กราฟ S11 และ S21 Model unit Cell



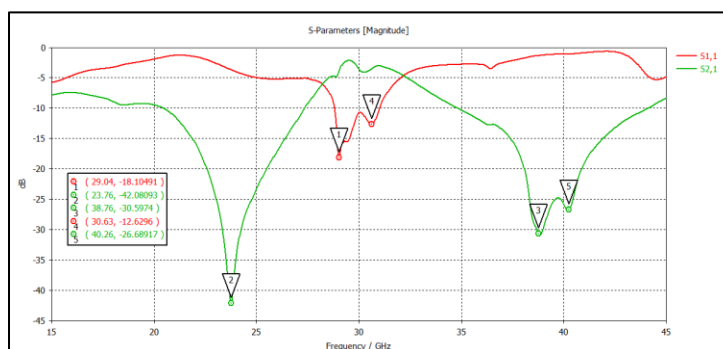
รูปที่ 4.3 กราฟ S11 และ S21 Model 2x2



รูปที่ 4.4 กราฟ S11 และ S21 Model 3x3



รูปที่ 4.5 กราฟ S11 และ S21 Model 4x4

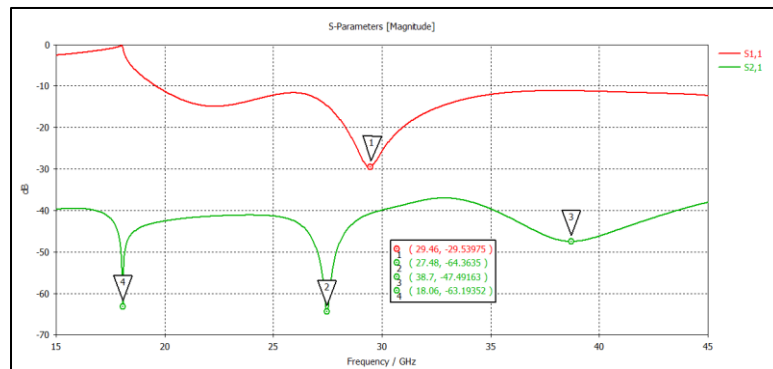


รูปที่ 4.6 กราฟ S11 และ S21 Model 5x5

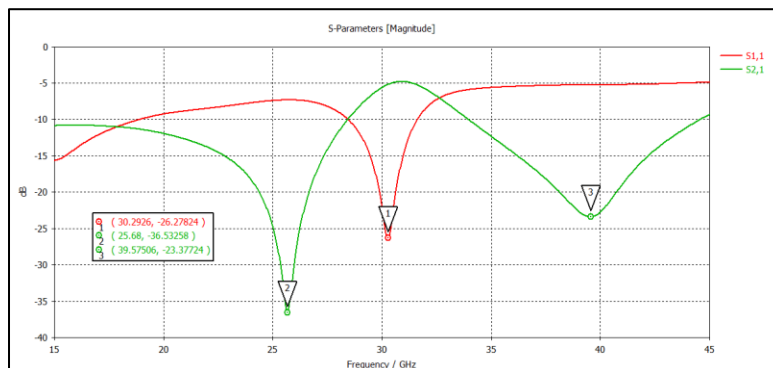
อีกทั้งยังมีการจำลอง Model 6x6, 7x7, 8x8, 9x9, 10x10 เพื่อหาค่า Tetrahedrons ที่สามารถนำมาอ้างอิงในการสร้างสมการทำนายเวลา โดยที่ไม่ได้มีการ Simulation ซึ่งค่า Tetrahedrons ที่ได้ คือ 981,844 1,330,090 1,735,784 2,194,696 และ 2,706,535 ตามลำดับ

3.7 ผลการจำลองที่ได้ แบบที่ใช้ Adaptive Mesh Refinement

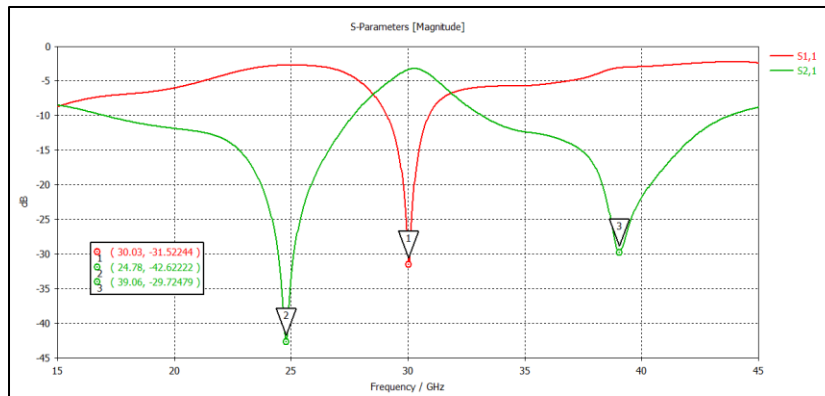
Model	Tetrahedrons	RAM	Time
Unit Cell	41,926	52%	60 วินาที
2x2	140,151	66%	4 นาที
3x3	244,285	72%	12 นาที
4x4	443,309	89%	26 นาที
5x5	795,771	96%	1 ชม. 2 นาที



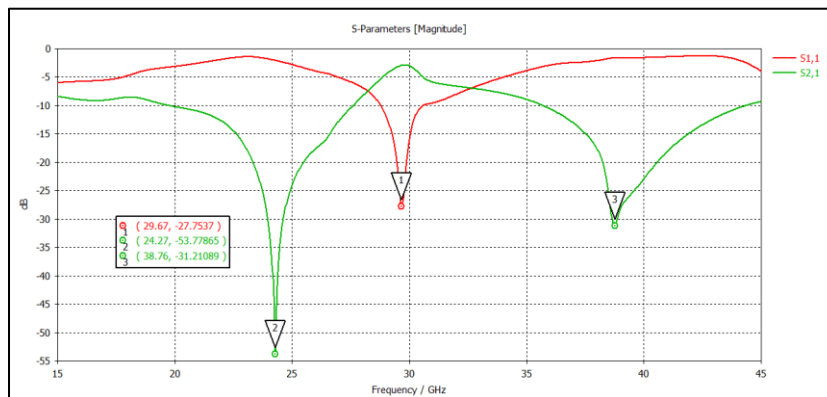
รูปที่ 4.7 กราฟ S11 และ S21 Model Unit Cell



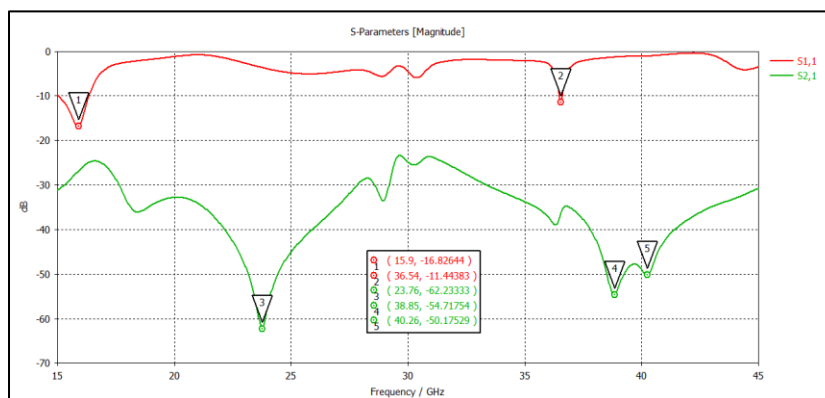
รูปที่ 4.8 กราฟ S11 และ S21 Model 2x2



รูปที่ 4.9 กราฟ S11 และ S21 Model 3x3



รูปที่ 4.10 กราฟ S11 และ S21 Model 4x4



รูปที่ 4.11 กราฟ S11 และ S21 Model 5x5

3.8 กราฟความสัมพันธ์และการสร้างสมการ (เฉพาะแบบที่ไม่ใช่ Adaptive Mesh Refinement)

ในการคำนวณหาสมการทำนายเวลาในการรันนี้ จะใช้ในเฉพาะแบบที่ไม่ใช่ Adaptive Mesh Refinement เท่านั้น เนื่องจากมีข้อมูลมากกว่า ซึ่งจะทำให้การสมการมีความน่าเชื่อถือมากกว่า อีกทั้งการรัน Simulation Full array ไม่เหมาะในการรันแบบ Frequency Domain Solve เนื่องจากมีการใช้ทรัพยากรที่มากและเวลานานโดยไม่จำเป็น

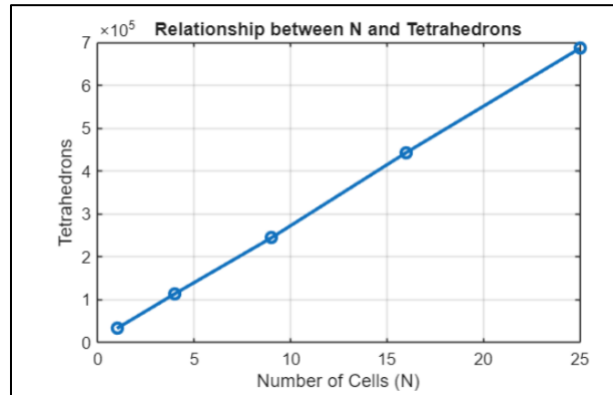
3.8.1 กราฟความสัมพันธ์

การสร้างกราฟความสัมพันธ์จะเป็นไปตามตารางที่ 2 ซึ่งผลดังกล่าวจะเป็นผลการจำลอง ณ ช่วงเวลานั้น เนื่องจากการคำนวณ Tetrahedrons จะไม่คงที่

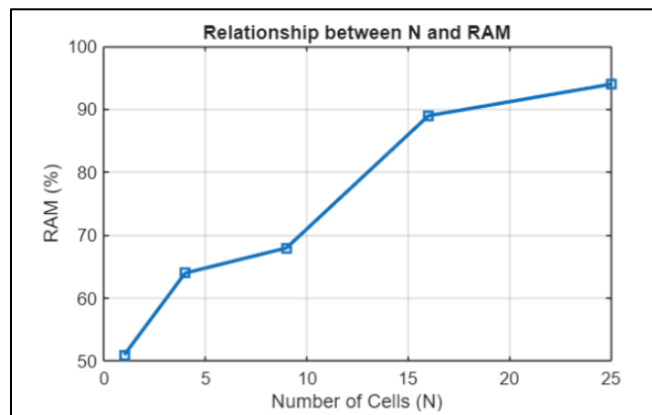
ตารางที่ 2

Model	Tetrahedrons	RAM	Time
Unit Cell	33,098	51%	38 วินาที
2x2	113,946	64%	3 นาที
3x3	244,285	68%	12 นาที
4x4	443,309	89%	26 นาที
5x5	686,401	94%	1 ชั่วโมง 13 นาที
6x6	981,844	-	-
7x7	1,330,090	-	-
8x8	1,735,784	-	-
9x9	2,194,696	-	-
10x10	2,706,535	-	-

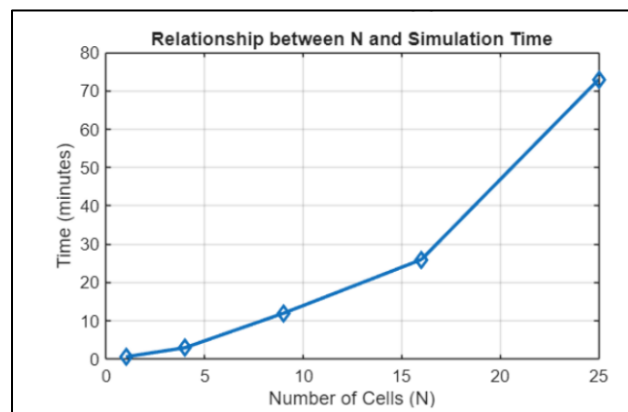
1. กราฟความสัมพันธ์ระหว่าง จำนวนเซลล์ (N) และ Tetrahedrons



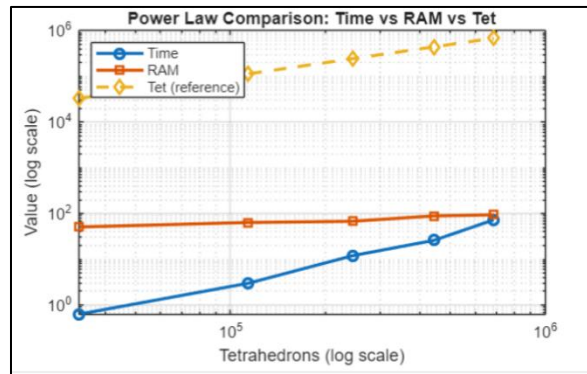
2. กราฟความสัมพันธ์ระหว่าง จำนวนเซลล์ (N) และ RAM(%)



3. กราฟความสัมพันธ์ระหว่าง จำนวนเซลล์ (N) และ Time



4. กราฟ Power Law



3.8.2 แนวคิดการสร้างสมการแบบ Dynamic Calibration

จากการสังเกตพบว่าการรัน Simulation แต่ละรอบ จำนวน Tetrahedrons ของโมเดลเดิมอาจมีค่าแตกต่างกันเล็กน้อยในแต่ละวัน เนื่องจากความคลาดเคลื่อนของการปัดเศษทศนิยม (Floating-point) และการประมวลผลแบบขนาน (Multi-threading) ดังนั้นจึงมีแนวคิดโดยใช้ค่า Tetrahedrons ของ Unit Cell ที่รันได้ในวันนั้นเป็นค่าฐาน (M_1) แล้วทำการประมาณค่าสำหรับ Array ขนาดใหญ่ต่อไป

3.8.3 สมการทำนายเวลาในการ Simulation

Model	N	Tetrahedrons (Tet)	Δ Tet จาก unit cell	Slope (Δ Tet/(N-1))
Unit Cell (1x1)	1	33,098	- (M_1)	-
2x2	4	113,946	+80,848	26,949
3x3	9	244,285	+211,187	26,398
4x4	16	443,309	+410,211	27,347
5x5	25	686,401	+653,303	27,221
6x6	36	981,844	+948,746	27,107
7x7	49	1,330,090	+1,296,992	27,021
8x8	64	1,735,784	+1,702,686	27,027
9x9	81	2,194,696	+2,161,598	27,020
10x10	100	2,706,535	+2,673,437	27,004
ค่าเฉลี่ย slope (ใช้เป็นค่าคงที่ในสมการ)				$\approx 27,000$

1. สมการทำนายจำนวน Tetrahedrons

จากการวิเคราะห์ข้อมูล พบว่าเมื่อขนาด Array เพิ่มขึ้น 1 element จำนวน Tetrahedrons จะเพิ่มขึ้นเฉลี่ยประมาณ 27,000 Tetrahedrons ต่อ element ดังนั้นสมการที่ได้คือ

$$Tet(N) = M_1 + 27,000 \times (N - 1) \quad (1)$$

โดยที่

- $Tet(N)$ คือจำนวน Tetrahedrons ที่ประมาณ
- M_1 คือจำนวน Tetrahedrons ของ Unit Cell ที่รันได้ในวันนั้น
- N คือจำนวน Unit Cell ทั้งหมดบน Model เช่น Model ขนาด 10×10 จะมี $N = 100$

2. สมการทำนายเวลา

นำค่า $Tet(N)$ จากสมการที่ (1) มาคำนวณเวลาประมวลผล โดยใช้ Power Law ที่ได้จากการ Regression กับข้อมูลจริง 5 จุด (Unit Cell ถึง 5×5)

$$Time(minute) = 5.75 \times 10^{-8} \times [Tet(N)]^{1.54} \quad (2)$$

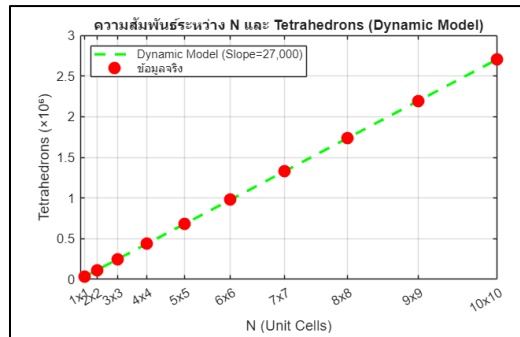
3. สมการทำนาย RAM

$$RAM(\%) = 5.95 \times [Tet(N)]^{0.20} \quad (3)$$

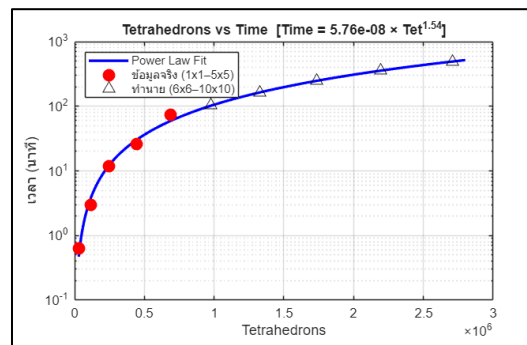
ซึ่งเป้าหมายในการสร้างสมการนี้คือหากต้องการรัน Model 20×20 จะต้องใช้เวลาเท่าไร เมื่อทำการคำนวณตามสมการนี้แล้วจะพบว่าจะต้องใช้เวลาในการSimulation ทั้งหมด 65.07 ชั่วโมง โดยมี Tetrahedrons = 10,206,098 ทั้งหมดเป็นการเทียบโดยใช้คอมพิวเตอร์ที่ใช้ในการทดสอบ หากทำการรันจริงจะต้องใช้RAMไป 151.79% ซึ่งเป็นไปไม่ได้

3.9 การตรวจสอบ (Verification)

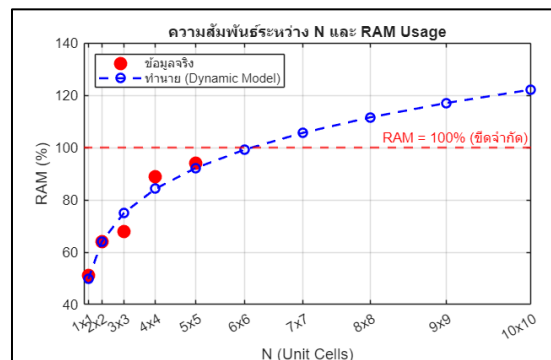
การเทียบผลทดสอบจะเป็นการเทียบจากผลทดสอบจริงตั้งแต่ unit cell ไปจนถึง 5x5 ที่มีการทดสอบจริง ซึ่งค่าที่นำมาเปรียบเทียบคือค่าที่ได้จากการคำนวณตามสมการที่ (1), (2), และ (3) โดยใช้ค่า $M1 = 33,098$ ได้ผลตามกราฟดังนี้



รูปที่ 4.12 กราฟความสัมพันธ์ระหว่าง N และ $Tet(N)$



รูปที่ 4.13 กราฟความสัมพันธ์ระหว่าง $Time(minute)$ และ $Tet(N)$



รูปที่ 4.14 กราฟความสัมพันธ์ระหว่าง $RAM(\%)$ และ $Tet(N)$

4. Time Domain Solve

ในการจำลองและวิเคราะห์โครงสร้างสายอากาศแบบแถวลำดับเต็มรูปแบบขนาด การประยุกต์ใช้ตัวแก้ปัญหาลำดับเต็มรูปแบบโดเมนเวลา (Time Domain Solver) ได้รับความนิยมและถือเป็นวิธีที่เหมาะสมที่สุด ซึ่งมีจะแผนการดำเนินงานดังนี้

1. ศึกษาหลักการของ Hexahedral Mesh (FIT), Time Step และ Accuracy Threshold
2. ปรับแต่งพารามิเตอร์หาค่าสมมูลระหว่างความแม่นยำ (Mesh/Accuracy) กับเวลาประมวลผล พร้อมประยุกต์ใช้ GPU Acceleration
3. ขยายสเกลจำลอง โดยจำลองจาก Unit Cell สู่ Full Array (เช่น 20×20)
4. วิเคราะห์ค่า S-Parameter และ Far-field Pattern ที่ย่านความถี่ 24, 30 และ 38 GHz
5. ตรวจสอบเปรียบเทียบผลลัพธ์และทรัพยากรกับ Frequency Domain Solver เพื่อสรุปความคุ้มค่าเชิงปฏิบัติ

4.1 หน้าที่และโครงข่ายที่ใช้ (Mesh Type)

ตัวแก้สมการแบบ Time Domain หรือ Transient Solver มีหน้าที่วิเคราะห์และคำนวณค่าพารามิเตอร์ S (S-parameters) และจำลองปัญหาสายอากาศได้ทุกรูปแบบ โดยมีจุดเด่นคือสามารถหาผลลัพธ์ครอบคลุมตลอดย่านความถี่กว้าง(Broadband) ได้จากการรันSimulationเพียงครั้งเดียว [4]โครงข่ายและเทคโนโลยีเป็นการทำงานแบบ Finite Integration Technique(FIT) โดยแบ่งพื้นที่การคำนวณด้วยตารางโครงข่ายรูปทรงสี่เหลี่ยมลูกบาศก์ที่เรียกว่า Hexahedral Mesh [2]

4.2 ความสัมพันธ์และทรัพยากร

1. การเพิ่มขึ้นของทรัพยากรแบบเชิงเส้นคือ ความต้องการด้านทรัพยากรการคำนวณและหน่วยความจำ(RAM)จะเพิ่มขึ้นในอัตราส่วนแบบเชิงเส้นตามขนาดของโมเดลหรือจำนวนเซลล์[3]
2. การประมวลผลขนาดใหญ่และฮาร์ดแวร์ ด้วยการจัดการหน่วยความจำที่มีประสิทธิภาพ ทำให้สามารถรันโมเดลขนาดใหญ่หลายๆ ที่มีตัวแปรไม่ทราบค่าระดับ หลายร้อยล้านตัวบนคอมพิวเตอร์พีซีระดับทั่วไปได้ [4] และยังรองรับการใช้ GPU ช่วยเร่งความเร็วเพื่อลดเวลาในการประมวลผลลงได้อย่างมาก [3]

4.3 ความสัมพันธ์ของเวลาที่ใช้รัน

- Time Step หากเพิ่มความละเอียดของ Mesh เซิงพื้นที่เช่น Lines/Cells per wavelength จะส่งผลให้โปรแกรมต้องลด Time Step ลงตามเงื่อนไขทางคณิตศาสตร์ ซึ่งจะใช้เวลาในการจำลองเพิ่มสูงขึ้นอย่างมีนัยสำคัญ [2]
- การจำลองด้วย Time Domain Solver จะสิ้นสุดการประมวลผลก็ต่อเมื่อระดับพลังงานสะสมที่ตกค้างในโดเมนการคำนวณ ลดลงต่ำกว่าเกณฑ์ความแม่นยำ(Accuracy Threshold)ที่กำหนดไว้

4.4 ความเหมาะสมในการนำไปใช้งาน

- เหมาะกับโมเดลที่มีขนาดใหญ่ทางไฟฟ้า เนื่องจากทรัพยากรเพิ่มขึ้นแค่แบบเส้นตรง จึงมีความเหมาะสมกับงานระดับ Full Phase Array [3]
- เหมาะกับการวิเคราะห์ผลกระทบในสภาพแวดล้อมจริง ซึ่งเหมาะสำหรับการประเมินโมเดลที่มีผลกระทบแบบไม่เป็นคาบ เช่น การวิเคราะห์ผลกระทบจากขอบอาร์เรย์(Edge effect), ผลกระทบจากเส้นจำลองการปรับเทียบ(Calibration lines) หรือการครอบอาร์เรย์ด้วยฝาครอบเรโดม(Radom) ที่มีพื้นผิวโค้ง [3]
- เหมาะกับงานที่ต้องการความถี่กว้าง สำหรับโครงสร้างที่ไม่มีความก้องกังวานสูงจนเกินไปและต้องการดูการตอบสนองของความถี่ในย่านที่กว้างมากๆ เพราะประหยัดเวลามากกว่าการรันแก้สมการที่ละจุดความถี่ [4]

4.5 การ Setting

- การตั้งค่าพื้นฐาน: Z Distance = 7.5
- Boundaries: Open (Add Space)
- Mesh: Hexahedral
- ในการรันใช้แค่ port 1

4.6 ข้อมูลที่ใช้ในการสร้างสมการ

ข้อมูลที่ใช้ในการสร้างแบบจำลองมาจากการทดสอบ 2 ชุด ได้แก่:

- การทดสอบค่า Accuracy ที่ระดับ Unit Cell (ตั้งแต่ -20 ถึง -80 dB)
- การทดสอบขนาด Array ตั้งแต่ 1×1 ถึง 10×10 ที่ Accuracy = -50 dB

เป้าหมายหลักคือการสร้างสมการรวม $T_{\text{total}}(A, n)$ ที่แมปความสัมพันธ์ระหว่างค่า Accuracy (A) และขนาด Array (n) กับเวลาการคำนวณรวมทั้งหมด

4.7 ผลการทดสอบ (Experimental Results)

1. การทดสอบ Accuracy ที่ระดับ Unit Cell

ทดสอบโดยการเปลี่ยนค่า Accuracy ตั้งแต่ -20 ถึง -80 dB โดยคงค่าพารามิเตอร์อื่นคงที่ วัดผลกระทบต่อ Run Time, RAM และ Mesh Cells

Accuracy(dB)	Run Time (s)	RAM (%)	Mesh Cells
-20	44	44-46	53,136
-25	45	46	53,136
-30	45	46	53,136
-35	45	46	53,136
-40	46	46	53,136
-50	48	46	53,136
-60	60	46	53,136
-80	116	46	53,136

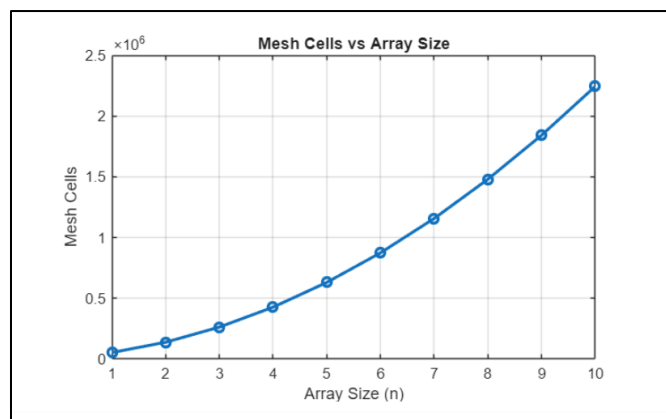
2. การทดสอบขนาด Array (Accuracy = -50 dB)

ทดสอบโดยเพิ่มขนาด Array ตั้งแต่ 1×1 ถึง 10×10 ที่ Accuracy = -50 dB คงที่ เพื่อวัดความสัมพันธ์ระหว่างขนาด Array กับทรัพยากรและเวลา

ขนาด Array	Elements	Run Time (s)	RAM (%)	Mesh Cells
1×1 (Unit Cell)	1	48	46	53,136
2×2	4	135	46	137,924
3×3	9	252	47	262,400
4×4	16	273	48	426,564
5×5	25	419	52	630,416
6×6	36	528	51	873,956
7×7	49	688	55	1,157,184
8×8	64	880	53	1,480,100
9×9	81	1,066	57	1,842,704
10×10	100	1,292	58	2,244,996

4.8 การสร้างสมการทางคณิตศาสตร์

4.8.1 แบบจำลองการขยายตัวเชิงพื้นที่: ขนาด Array กับจำนวน Mesh Cells เมื่อนำค่าจากการทดสอบ เพื่อหาความสัมพันธ์กันทางคณิตศาสตร์จะได้ผลกราฟดังนี้



รูปที่ 1 กราฟความสัมพันธ์ ขนาด Array กับจำนวน Mesh Cells

จากรูปที่ 1 จากการขยาย Array ส่งผลให้ปริมาตรทางกายภาพเพิ่มขึ้นในเชิงกำลังสอง แต่เนื่องจากกรอบขอบเขต (Bounding Box) มีระยะห่างคงที่สำหรับ Open Boundary จำนวน Mesh Cells จึงไม่ Scale แบบ n^2 อย่างสมบูรณ์ จึงเสนอสมการพหุนามอันดับสอง:

$$M(n) = \alpha_1 \cdot n^2 + \alpha_2 \cdot n + \alpha_3$$

แทนค่าจากข้อมูลทดสอบ $n=1$, $n=5$, และ $n=10$ แล้วแก้ระบบสมการเชิงเส้น:

n	M(n) จากข้อมูล	สมการ
$n = 1$	53,136	$\alpha_1 + \alpha_2 + \alpha_3 = 53,136$
$n = 5$	630,416	$25\alpha_1 + 5\alpha_2 + \alpha_3 = 630,416$
$n = 10$	2,244,996	$100\alpha_1 + 10\alpha_2 + \alpha_3 = 2,244,996$

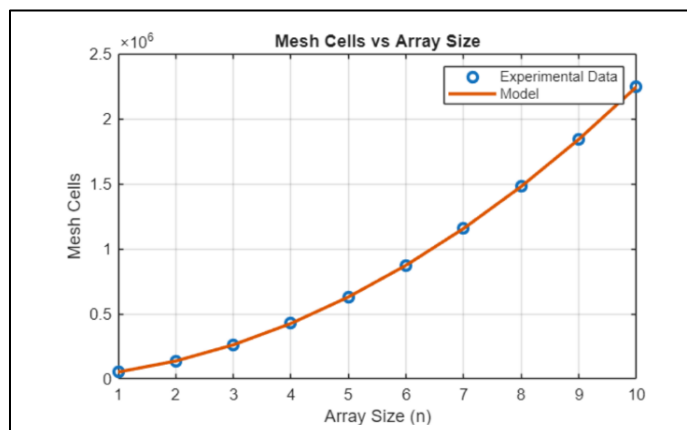
แก้ระบบสมการได้ค่าสัมประสิทธิ์:

- $\alpha_1 = 19,844$ (จำนวน Cell ที่ขึ้นอยู่กับพื้นที่ภายใน)
- $\alpha_2 = 25,256$ (จำนวน Cell ที่ขึ้นอยู่กับเส้นรอบวง/Padding)
- $\alpha_3 = 8,036$ (ค่าคงที่สำหรับมุมและ Fixed Overhead)

ดังนั้น สมการการสร้าง Mesh คือ:

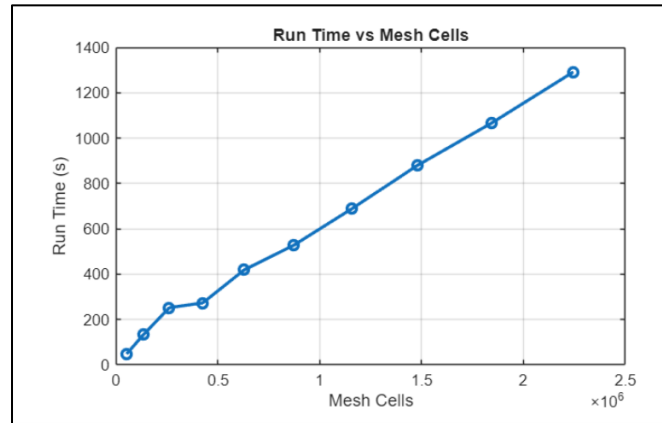
$$M(n) = 19,844 \cdot n^2 + 25,256 \cdot n + 8,036$$

การตรวจสอบ: ทำการพล็อตกราฟเทียบข้อมูลจริงจะได้ดังรูปที่ 2



รูปที่ 2 กราฟสมการการสร้าง mesh และค่าจริง

4.8.2 ความซับซ้อนเชิงคำนวณ: Mesh Cells กับเวลาฐาน



รูปที่ 3 กราฟความสัมพันธ์ Mesh Cells กับ Run Time

โปรแกรม FDTD (Finite-Difference Time-Domain) แสดงเวลาคำนวณที่ Scale เชิงเส้นตรงกับจำนวน Mesh Cells สำหรับ Time-step และ Pulse Count คงที่ โดยนิยามรูปแบบ:

$$T_{base(n)} = (\beta \cdot M(n)) + T_{setup}$$

โดยที่ T_{setup} คือเวลาคงที่สำหรับ Initialization และ β คือเวลาต่อการประมวลผล 1 Cell

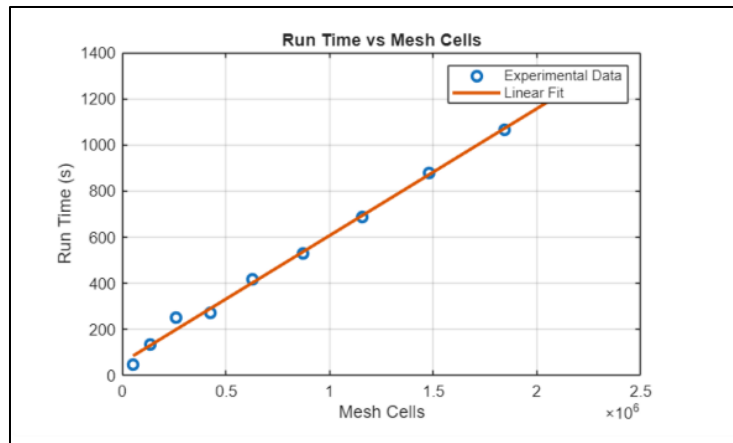
แทนค่าจากจุดสุดขีด ($n=1$, $T=48$ วินาที) และ ($n=10$, $T=1,292$ วินาที):

$$1,292 = \beta(2,244,996) + T_{setup}$$

$$48 = \beta(53,136) + T_{setup}$$

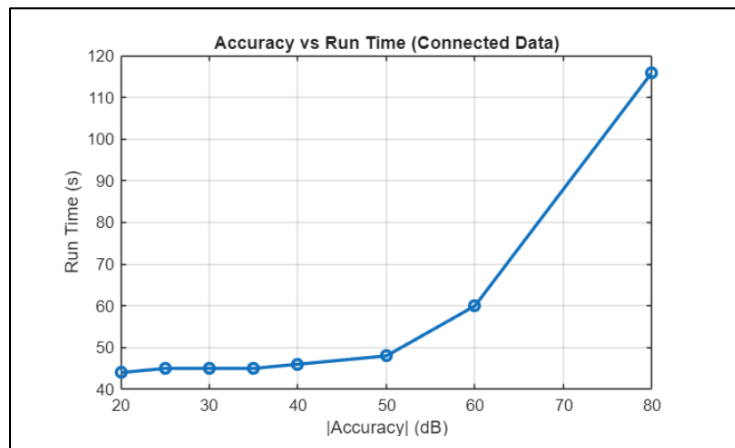
แก้สมการได้: $\beta \approx 5.67 \times 10^{-4}$ วินาที/Cell และ $T_{setup} \approx 18$ วินาที

การตรวจสอบ: ทำการพล็อตกราฟเทียบข้อมูลจริงจะได้ดังรูปที่ 4



รูปที่ 4 กราฟเปรียบเทียบสมการการที่ () และค่าจริง

4.8.3 การบรรจุเชิงเวลา: Accuracy กับ Solver Time (Penalty Function)



รูปที่ 5 กราฟความสัมพันธ์ |Accuracy ใน dB| กับ Run Time

ค่า Accuracy ใน CST กำหนด Energy Decay Threshold ที่จำเป็นต้องถึงก่อนหยุดการคำนวณ เมื่อตั้งค่าเข้มงวดขึ้น Solver ต้องประมวลผลมากขึ้น

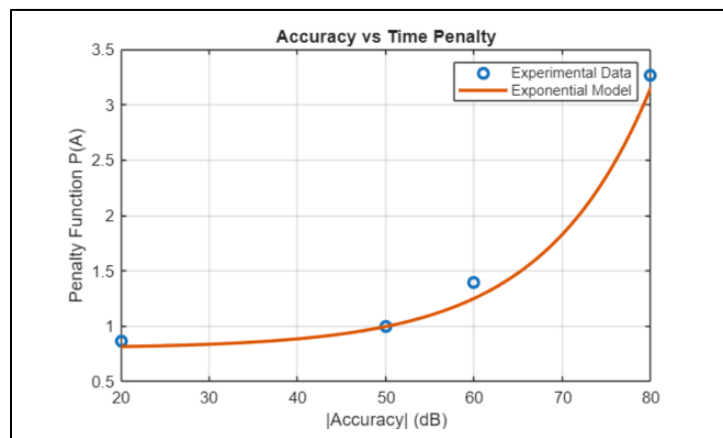
กำหนด $A = |\text{Accuracy ใน dB}|$ และนิยาม Penalty Function $P(A)$ แทนตัวคูณบนเวลา Solver ฐาน (โดย $P(50) \approx 1$):

จากข้อมูล Unit Cell หักเวลา Setup ออก ($T_{\text{solve}} = T_{\text{total}} - 18$):

A (dB)	T_{total} (s)	T_{solve} (s)	$P(A) = T_{\text{solve}} / T_{\text{solve}(50)}$
20	44	26	0.867
50	48	30	1.000
60	60	42	1.400
80	116	98	3.267

Fitting เส้นโค้งแบบ Exponential สะท้อนลอการิทึมของ Energy Threshold จะได้สมการดังนี้:

$$P(A) = 0.8 + 0.0032 \cdot e^{0.0825 \cdot A}$$



รูปที่ 6 กราฟเปรียบเทียบสมการการที่ () และค่าจริง

4.8.4 สมการทำนายรวม (Unified Predictive Model)

รวมแบบจำลองการขยายตัวเชิงพื้นที่และการบรรจบเชิงเวลา ได้สมการหลักในการทำนายเวลาดังนี้

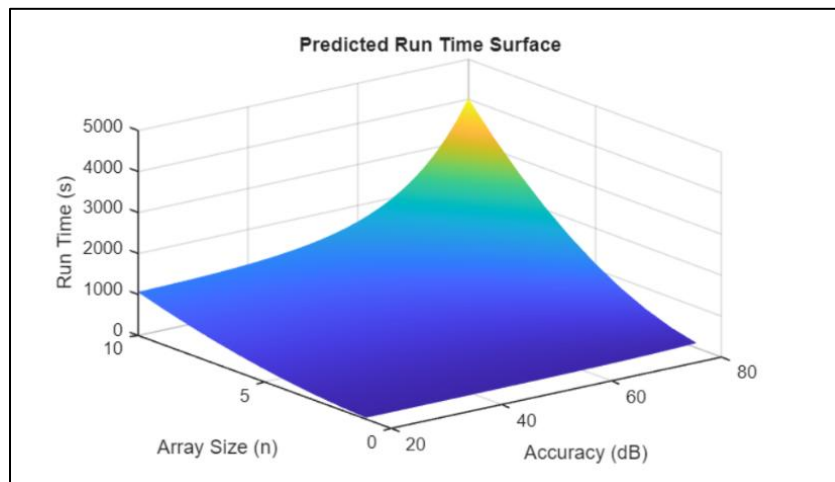
$$T_{\text{total}(A,n)} = T_{\text{setup}} + (P(A) \cdot \beta \cdot M(n))$$

แทนค่าตัวแปรทั้งหมด ได้สมการหลักสมบูรณ์:

$$T_{\text{total}(A,n)} = 18 + [(0.8 + 0.0032 \cdot e^{0.0825 \cdot A})(5.67 \times 10^{-4}) (19,844 \cdot n^2 + 25,256 \cdot n + 8,036)]$$

ตารางสรุปพารามิเตอร์ของสมการ

ตัวแปร	ค่าในสมการ	ความหมาย
$T_{total}(A, n)$	ผลลัพธ์ (วินาที)	เวลาประมวลผลรวมทั้งหมด
T_{setup}	18 วินาที	Overhead เริ่มต้น: สร้างเมทริกซ์, กำหนดขอบเขต, คำนวณ Port Modes
n	ตัวแปรอิสระ	จำนวน Hexahedral Mesh Cells ทั้งหมดในโดเมนการคำนวณ
β (เบตา)	5.67×10^{-4} วินาที/เซลล์	ค่าคงที่ประสิทธิภาพฮาร์ดแวร์ เวลาเฉลี่ยต่อเซลล์ต่อรอบ
A	หน่วย dB (ค่าสัมบูรณ์)	ระดับความแม่นยำเป้าหมาย เช่น 30, 40, 50, 60 dB
$P(A)$	ฟังก์ชัน Exponential	ตัวคูณปรับโทษจากการสลายพลังงาน เพิ่มขึ้นตาม A



รูปที่ 7 กราฟสมการทำนายรวม

4.9 การตรวจสอบและวิเคราะห์ความคลาดเคลื่อน (Validation & Error Analysis)

เพื่อตรวจสอบความถูกต้องของสมการ ได้ทำการเปรียบเทียบค่าทำนายกับข้อมูลทดสอบจริงในหลายกรณี:

กรณีทดสอบ	เวลาจริง	เวลาทำนาย	ความคลาดเคลื่อน (%)
Unit Cell, -20 dB	44 s	42.5 s	-3.4%
Unit Cell, -60 dB	60 s	55.6 s	-7.3%
Array 4×4, -50 dB	273 s	259.0 s	-5.1%
Array 8×8, -50 dB	880 s	857.0 s	-2.6%
Array 10×10, -50 dB	1,292 s	1,290.0 s	-0.15%

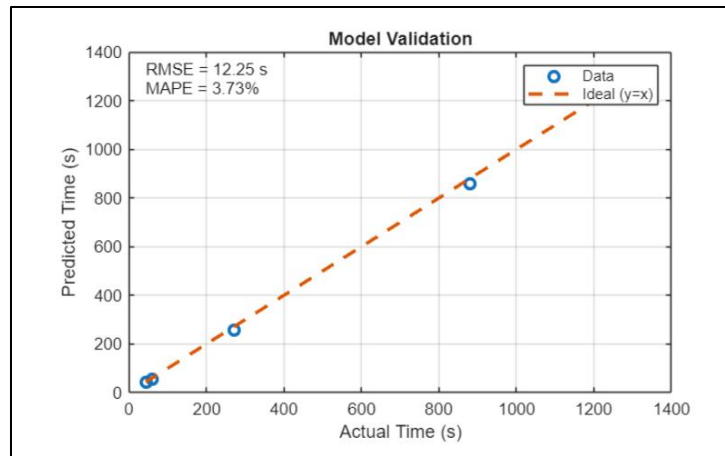
เพื่อประเมินความแม่นยำของแบบจำลองเชิงทำนาย ได้ทำการเปรียบเทียบค่าที่คำนวณได้จากสมการกับค่าที่ได้จากการทดลองจริง โดยพล็อตกราฟระหว่างค่าจริง (Actual Time) และค่าที่แบบจำลองทำนาย (Predicted Time) ในการประเมินประสิทธิภาพของแบบจำลองในเชิงปริมาณ ได้ใช้เมตริกความคลาดเคลื่อนมาตรฐาน 2 ตัว ดังนี้

Root Mean Square Error (RMSE): วัดค่าความคลาดเคลื่อนในหน่วยเดียวกับเวลา (วินาที) ไวต่อความคลาดเคลื่อนขนาดใหญ่

$$RMSE = \sqrt{\left(\frac{1}{n}\right) \cdot \sum (T_{pred,i} - T_{actual,i})^2}$$

Mean Absolute Percentage Error (MAPE): วัดความคลาดเคลื่อนในรูปเปอร์เซ็นต์ เปรียบเทียบได้ข้ามช่วงเวลาที่แตกต่างกัน

$$MAPE = \frac{1}{n} \cdot \sum \frac{|T_{pred,i} - T_{actual,i}|}{T_{actual,i}} \times 100\%$$



รูปที่ 8 Model Validation

จากการคำนวณพบว่า ค่า RMSE มีค่าประมาณ 12.25 วินาที ซึ่งแสดงให้เห็นว่าค่าที่แบบจำลองทำนายมีความคลาดเคลื่อนเฉลี่ยในระดับต่ำเมื่อเทียบกับช่วงเวลาการคำนวณทั้งหมด ขณะที่ค่า MAPE มีค่าประมาณ 3.73% ซึ่งบ่งชี้ว่าแบบจำลองสามารถทำนายค่าได้อย่างแม่นยำในเชิงสัดส่วน

นอกจากนี้ จากกราฟพบว่าจุดข้อมูลส่วนใหญ่อยู่ใกล้กับเส้นอุดมคติ ($y = x$) แสดงให้เห็นถึงความสอดคล้องระหว่างค่าที่ทำนายและค่าจริง โดยมีความคลาดเคลื่อนสูงสุดไม่เกินประมาณ 7.3% ซึ่งยังอยู่ในเกณฑ์ที่ยอมรับได้สำหรับการประมาณค่าทางวิศวกรรม

ดังนั้น สมการทำนายเวลาที่พัฒนาขึ้นสามารถเขียนในรูปแบบที่รวมค่าความคลาดเคลื่อนได้เป็น

$$T_{total}(A, n) = [T_{setup} + (P(A) \cdot \beta \cdot M(n))] \times (1 \pm 3.73\%)$$

โดยค่า MAPE เท่ากับ 3.73% แสดงถึงความคลาดเคลื่อนเฉลี่ยของโมเดล และพบว่าความคลาดเคลื่อนสูงสุดไม่เกินประมาณ 7.3% ซึ่งยังอยู่ในเกณฑ์ที่ยอมรับได้สำหรับการประมาณค่าในครั้งนี้

5. การสร้างสมการทำนายแบบจำลองแบบทั่วไป (Time Domain Solve)

5.1 ข้อจำกัดของแบบจำลองดั้งเดิม

แบบจำลองการทำนายเวลาในรูปแบบดั้งเดิมมักสร้างขึ้นโดยผู้สมการเข้ากับตัวแปรทางเรขาคณิตเฉพาะของชิ้นงาน เช่น ขนาด Array ของ Metasurface (M×N) หรือขนาดกายภาพของโครงสร้าง แนวทางนี้ส่งผลให้เกิดข้อจำกัดพื้นฐาน 3 ประการ ได้แก่

- สมการที่ได้ใช้ได้เฉพาะกับโครงสร้างรูปทรงเดิมที่ใช้สร้างสมการเท่านั้น
- หากเปลี่ยนวัสดุ ขนาด หรือรูปทรงของโครงสร้าง จำเป็นต้องสร้างสมการใหม่ทั้งหมด
- ไม่สามารถนำความรู้จากโครงสร้างหนึ่งไปประยุกต์กับอีกโครงสร้างหนึ่งได้

5.2 แนวคิดการแทนที่ด้วยความซับซ้อนของโครงข่าย

เพื่อแก้ไขข้อจำกัดข้างต้น แบบจำลองที่นำเสนอในรายงานนี้ได้ตัดความสัมพันธ์ที่ยึดติดกับตัวแปรทางรูปทรง (Geometry-dependent Variables) ออกทั้งหมด และแทนที่ด้วยตัวแปรสากลเพียงตัวเดียว คือ จำนวนเซลล์โครงข่ายทั้งหมด (Total Mesh Cells, N_{cells}) โดยมีแบบจำลองทำนายเวลาประมวลผลรวม T_{total} ประกอบด้วยตัวแปรอิสระ 2 ตัวหลัก ได้แก่ จำนวนเซลล์โครงข่าย (N_{cells}) และระดับความแม่นยำเป้าหมาย (A หน่วยเป็น dB) โดยมีรูปแบบสมการทั่วไปดังนี้

$$T_{total}(A, N_{cells}) = [T_{setup} + (P(A) \cdot \beta \cdot N_{cells})](1 \pm 3.73\%)$$

เหตุผลที่ N_{cells} เป็นตัวแทนที่เหมาะสมสำหรับแบบจำลองสากล มีพื้นฐานมาจากคุณสมบัติของอัลกอริทึม FDTD ดังนี้

- อัลกอริทึม FDTD อัปเดตสนามแม่เหล็กไฟฟ้าทุกเซลล์ในทุกขั้นเวลา (Time Step) ทำให้ภาระงานคำนวณแปรผันโดยตรงกับจำนวนเซลล์
- โดยไม่ว่าโครงสร้างจะมีรูปทรงเป็นวงกลม สี่เหลี่ยม หรือรูปร่างซับซ้อนใด ๆ ก็ตาม N_{cells} จะสะท้อนภาระงานคำนวณที่แท้จริงได้เสมอ
- N_{cells} สามารถหาค่าได้ง่ายจากโปรแกรม CST โดยไม่ต้องรันการจำลองจริง ทำให้เหมาะสมอย่างยิ่งสำหรับการประเมินเบื้องต้น

ดังนั้นการเปลี่ยนจาก Geometry Variables $\rightarrow$ N_cells จึงทำให้สมการกลายเป็นแบบจำลองที่สามารถนำไปใช้ทำนายเวลาประมวลผลของโครงสร้างรูปทรงใด ๆ ก็ได้ โดยไม่ต้องปรับสมการใหม่

5.3 วิธีดำเนินการ (Methodology)

5.3.1 การหาค่า N_cells จากโปรแกรม CST โดยไม่ต้องรันการจำลอง

ข้อได้เปรียบสำคัญของแบบจำลองนี้คือ ไม่จำเป็นต้องรันการจำลองเต็มรูปแบบเพื่อหาค่า N_cells วิศวกรสามารถหาค่านี้จากโปรแกรม CST ได้ด้วยขั้นตอน 4 ขั้นตอน ดังนี้

1. สร้างโมเดล 3 มิติของโครงสร้างที่ต้องการในโปรแกรม CST Studio Suite
2. ไปที่เมนู Mesh แล้วเลือกคำสั่ง "Update Mesh" (สังเกตว่าเป็นคำสั่งสร้าง Mesh เท่านั้น ไม่ใช่การ Start Simulation)
3. โปรแกรมจะประมวลผลการสร้างโครงข่ายเชิงพื้นที่ แล้วแสดงผล "Total number of mesh cells" ในหน้าต่าง Message Window
4. นำตัวเลขจำนวนเต็มดังกล่าวมาแทนค่าในตัวแปร N_cells ในสมการทั่วไป

กระบวนการสร้าง Mesh นี้ใช้เวลาเพียงไม่กี่วินาทีถึงไม่กี่นาที ขึ้นอยู่กับความซับซ้อนของโครงสร้าง เมื่อเทียบกับเวลาจำลองจริงที่อาจใช้หลายชั่วโมงหรือหลายวัน จึงถือว่าเป็นวิธีที่ประหยัดเวลาอย่างมีนัยสำคัญ

5.3.2 การปรับเทียบสำหรับฮาร์ดแวร์หรือโครงสร้างใหม่ (Calibration Strategy)

หากมีการเปลี่ยนแปลงสำคัญในระบบ เช่น การเปลี่ยนฮาร์ดแวร์ประมวลผล หรือการทำงานกับวัสดุที่มีค่า Resonance สูงมาก (ซึ่งทำให้พลังงานสลายตัวช้าลง) ค่า β อาจต้องปรับใหม่ โดยสามารถทำได้ด้วย 2 การทดสอบขนาดเล็ก

การทดสอบที่ 1 (Baseline Run): รันโมเดลขนาดเล็ก เช่น Unit Cell ที่ Accuracy ต่ำ (เช่น $A = 30$ dB) บันทึกค่า N_cells,1 และเวลาจริง T_1

การทดสอบที่ 2 (Scaling Run): รันโมเดลที่ขยายขนาดขึ้น เช่น Array 2x2 ที่ A เท่าเดิม บันทึกค่า N_cells,2 และ T_2

กำหนดให้ทำการจำลองสองครั้งบนฮาร์ดแวร์เดียวกัน โดยใช้โครงสร้างที่มีขนาดตาข่ายต่างกัน M_1 และ M_2 ($M_1 \neq M_2$) ได้เวลาประมวลผลที่วัดได้ T_1 และ T_2 ตามลำดับ

$$T_1 = T_{\text{setup}} + \beta \cdot N_{\text{cells}, 1}$$

$$T_2 = T_{\text{setup}} + \beta \cdot N_{\text{cells}, 2}$$

เนื่องจากเป็นระบบสมการเชิงเส้นสองสมการสองตัวแปร จึงสามารถแก้หา β ได้โดยการลบสมการ ดังนี้

$$\beta = \frac{(T_2 - T_1)}{(N_{\text{cells}, 2} - N_{\text{cells}, 1})}$$

และจากนั้นสามารถแก้หา T_{setup} ได้

$$T_{\text{setup}} = T_1 - \beta \cdot M_1$$

นำค่า T_{setup} ที่ได้ไปแทนในสมการที่สมการทั่วไป

จากนั้นคำนวณค่า β ที่ปรับเทียบแล้วด้วยสมการความชัน

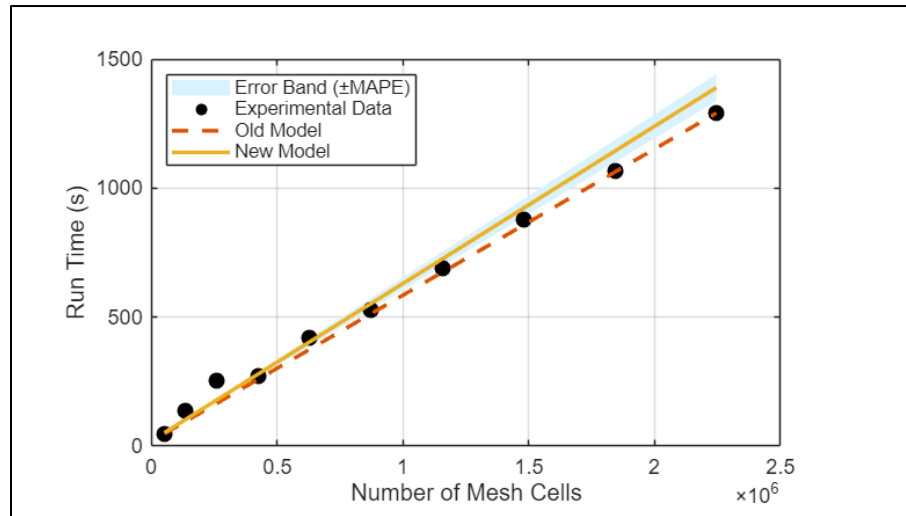
$$\beta_{\text{calibrated}} = \frac{(T_2 - T_1)}{P(A) \cdot (N_{\text{cells}, 2} - N_{\text{cells}, 1})}$$

นำค่า $\beta_{\text{calibrated}}$ ไปแทนในสมการที่สมการทั่วไป

ดังนั้นสมการสำหรับใช้งานต่อไป โดยฟังก์ชัน $P(A)$ ยังคงใช้ค่าเดิม เนื่องจากเป็นคุณสมบัติทางคณิตศาสตร์ภายในของ CST ที่ไม่ขึ้นกับฮาร์ดแวร์ จะได้สมการดังนี้

$$T_{\text{total}(A, N_{\text{cells}})} = [T_{\text{setup}} + (P(A) \cdot \beta_{\text{calibrated}} \cdot N_{\text{cells}})](1 \pm 3.73\%)$$

การตรวจสอบ: เมื่อนำสมการใหม่ที่สามารถใช้กับแผ่นสะท้อนทั่วไปได้มาเปรียบเทียบกับสมการที่ยึดแค่แผ่นสะท้อนเดียว จะได้ตามรูปที่ 9



รูปที่ 9 Old Model และ New Model

6. การตรวจสอบความถูกต้องกับโครงสร้าง Metasurface อื่น

$N_{\text{cells},1} = 82524$ -, $T_1 = 54$ s, $N_{\text{cells},2} = 213564$, $T_2 = 101$ s

คำนวณ β ได้ = 3.6×10^{-4}

คำนวณ $T_{\text{setup}} = 24.4$ s

กรณีทดสอบ	Ncells	เวลาจริง	เวลาทำนาย	ช่วงเวลายทำนาย ($\pm 3.73\%$)
Unit Cell	82,524	54 s	54.11 s	52.09 – 56.13 s
Scaling Run	213,564	101 s	101.28 s	97.51 – 105.06 s
Array 4×4	659,100	257 s	261.68 s	251.92 – 271.44 s
Array 8×8	2,283,996	740 s	846.64 s	815.06 – 878.22 s
Array 10×10	3,463,356	1,140 s	1,271.21 s	1,223.79 – 1,318.62 s